

Typography System — Vermont Health Platform

For: Developers, designers, content editors, and anyone touching the frontend codebase.

Why this exists: After the app grew to 200+ pages, typography had drifted — different pages used different font sizes for equivalent content, headings were larger on mobile than desktop, and a single style change required hunting through dozens of files. This document describes the system that was built to eliminate that problem permanently.

Table of Contents

- 1. The Core Problem This Solves
- 2. How Tailwind CSS Text Sizing Works
- 3. What Responsive Typography Means (and Why It Matters)
- 4. The globals.css File — Full Walkthrough
- 5. The Typography Scale Classes (ty-*)
- 6. How to Use the Typography Classes
- 7. What NOT to Do
- 8. How to Change the Scale App-Wide
- 9. Adding a New Typography Class
- 10. Color, Weight, and Spacing — The Rules
- 11. Dark Mode
- 12. Emoji and Icon Sizing — The Exception
- 13. Quick Reference Card

1. The Core Problem This Solves

Before this system, a developer writing a new page would type something like:

```
<h1 className="text-3xl font-black text-slate-900">Page Title</h1>
<p className="text-lg text-slate-600 leading-relaxed">Description here.</p>
```

Another developer on a different page would write:

```
<h1 className="text-4xl font-extrabold text-gray-900">Page Title</h1>
<p className="text-xl text-gray-600 leading-relaxed">Description here.</p>
```

Both pages intend to look the same. Neither does. And there is no way to enforce or audit consistency without manually reading every file.

The fix: Typography sizing is now defined in exactly one place — `app/globals.css`. Every page references the same named class. If you need to change the heading size across the entire app, you change one line. You do not touch 200 component files.

2. How Tailwind CSS Text Sizing Works

Tailwind CSS is a utility-first CSS framework. Instead of writing custom CSS classes, you apply small, single-purpose utility classes directly in your JSX.

The text size scale

Tailwind's text size classes map directly to CSS `font-size` values:

Class	Font Size	Line Height	Typical use
-------	-----------	-------------	-------------

Class	Font Size	Line Height	Typical use
<code>text-xs</code>	12px	16px	Labels, captions, badges
<code>text-sm</code>	14px	20px	Secondary body text
<code>text-base</code>	16px	24px	Primary body text
<code>text-lg</code>	18px	28px	Large body / small heads
<code>text-xl</code>	20px	28px	Sub-headings
<code>text-2xl</code>	24px	32px	Section headings
<code>text-3xl</code>	30px	36px	Page headings
<code>text-4xl</code>	36px	40px	Large page headings
<code>text-5xl</code>	48px	1	Marketing heroes
<code>text-6xl</code>	60px	1	Display / splash

The responsive prefix system

Tailwind uses **breakpoint prefixes** to apply styles at specific screen widths:

Prefix	Breakpoint	Screen width	Meaning
(none)	all	0px and up	Mobile-first (always active)
<code>sm:</code>	small	640px and up	Small tablets
<code>md:</code>	medium	768px and up	Tablets and larger
<code>lg:</code>	large	1024px and up	Desktops
<code>xl:</code>	extra-large	1280px and up	Large desktops

Critical rule: Tailwind is **mobile-first**. The unprefixed class is the *default* (mobile). Prefixed classes *override* at larger screens. You are always writing "start here on mobile, change to this on larger screens."

So `text-sm md:text-base` means:

- Mobile (< 768px): 14px text
- Tablet and desktop (≥ 768px): 16px text

And `text-2xl md:text-3xl lg:text-4xl` means:

- Mobile: 24px
- Tablet: 30px
- Desktop: 36px

3. What Responsive Typography Means (and Why It Matters)

The rule

- **Headings scale UP as screens get larger.** You start small on mobile and increase.
- **Body text scales UP as screens get larger.** You start at a readable mobile size and optionally increase on desktop for comfort.
- **Labels, badges, and metadata** usually stay fixed at `text-xs` or `text-sm` — no responsive scaling needed.

Why headings need to scale down on mobile

A `text-5xl` (48px) heading looks dramatic on a 1440px desktop. On a 375px iPhone, it takes up four lines and dominates the screen. The correct approach is to start at a size that fits the narrowest screen and grow from there.

Why flat sizes are a bug

Writing just `text-3xl` on a heading means:

- It's 30px on a 375px iPhone. That's often too large — it can overflow, truncate, or crowd out other content.
- It's still 30px on a 1440px desktop. That's often too small — it looks weak next to high-resolution displays.

Responsive sizing fixes both problems simultaneously.

The correct direction

```
WRONG: text-5xl           ← same on all screens, usually too big on mobile
WRONG: text-5xl md:text-3xl ← getting SMALLER on bigger screens (backwards)
RIGHT: text-2xl md:text-3xl lg:text-4xl ← starts small, grows up ✓
```

4. The globals.css File — Full Walkthrough

The file lives at `frontend/app/globals.css`. Every section is documented below.

`@import "tailwindcss"`

Loads the entire Tailwind v4 framework. This single line replaces the old `@tailwind base`, `@tailwind components`, `@tailwind utilities` directives from Tailwind v3. Everything Tailwind provides — reset styles, utility classes, responsive variants — comes from this import.

`@variant dark`

```
@variant dark (&:is(.dark *));
```

Tells Tailwind that dark mode is controlled by a `.dark` class on a parent element (typically `<html>`), not by the OS media query. The `ThemeProvider` component toggles this class. This means dark mode responds to the user's in-app preference toggle, not their system setting.

`@theme { ... }`

This is Tailwind v4's design token system. Variables defined here become usable as Tailwind utility classes automatically. For example:

```
@theme {
  --color-brand-policy: #0369a1;
}
```

This makes `text-brand-policy`, `bg-brand-policy`, and `border-brand-policy` available as Tailwind classes everywhere in the app — without writing any custom CSS.

What lives in `@theme`:

- **Z-index scale** — Named z-index values (`--z-modal: 60`) instead of arbitrary numbers. Use `z-modal` in JSX instead of `z-[60]`. If the modal z-index needs to change, one line here fixes every modal in the app.
- **Layout dimensions** — Structural measurements like `--sidebar-width: 18rem`. Components reference these CSS variables directly so a sidebar resize propagates everywhere.
- **Pillar brand colors** — The six content pillar colors (Policy, Economics, Technology, Clinical, Equity, Academy) plus Advisory. Used for pillar-specific badges, borders, and accents throughout the app.
- **UI colors** — Global interface colors for borders, backgrounds, and primary actions.

- **Surface colors** — Page background variants (light, muted, hero, dark). These are the building blocks of section backgrounds.

`.dark { ... }`

Overrides the CSS variables from `@theme` when dark mode is active. This is how dark mode works in this app — it does not duplicate Tailwind utility classes everywhere. Instead, the design tokens change, and components using those tokens automatically update.

`:focus-visible` and `:focus:not(:focus-visible)`

Accessibility rules. `:focus-visible` shows a keyboard focus ring (required by WCAG 2.4.7 for keyboard navigation). `:focus:not(:focus-visible)` hides it for mouse clicks so it doesn't appear during normal mouse interaction. **Do not remove these.**

`@layer components { .ty-* }` — The Typography Scale

The typography system. Documented in full in the next section.

`@layer utilities { .animate-marquee }`

A custom animation for the ticker strip. `@layer utilities` is for single-purpose helpers that behave like Tailwind utilities (they can be combined with responsive prefixes, hover states, etc.).

`@media print`

Hides navigation, sidebars, and headers when a page is printed. Ensures article content prints cleanly. Do not remove.

5. The Typography Scale Classes (ty-*)

These six classes are the **entire typography sizing system** for the app. Every responsive text size in every page and component flows through one of these six classes.

```
@layer components {
  .ty-h1      { @apply text-2xl md:text-3xl lg:text-4xl; }
  .ty-h1-xl   { @apply text-2xl md:text-4xl lg:text-5xl; }
  .ty-h2      { @apply text-xl  md:text-2xl lg:text-3xl; }
  .ty-h3      { @apply text-lg   md:text-xl;           }
  .ty-hero    { @apply text-base md:text-lg;           }
  .ty-body    { @apply text-sm   md:text-base;         }
}
```

What each class is for

`.ty-h1` — Page headings

The primary `<h1>` on a page. Every major page in the platform uses this: policy hub, economics hub, advisory pages, academy, research lab, states, etc.

```
Mobile (< 768px): 24px — fits cleanly on a 375px iPhone
Tablet (≥ 768px): 30px — comfortable on an iPad
Desktop (≥ 1024px): 36px — authoritative on a laptop/monitor
```

`.ty-h1-xl` — Large marketing hero headings

For pages that are intentionally dramatic and have full-width hero sections: values page, FAQ, HTR simulator intro, large landing areas. Slightly more aggressive scaling.

Mobile	(< 768px):	24px
Tablet	(≥ 768px):	36px
Desktop	(≥ 1024px):	48px

.ty-h2 — Section headings within a page

Used for major sections inside a long page — "How It Works", "Pricing", "Research Capabilities", etc. Also used for secondary page titles like within tabs.

Mobile	(< 768px):	20px
Tablet	(≥ 768px):	24px
Desktop	(≥ 1024px):	30px

.ty-h3 — Card and subsection headings

The title inside a card, a feature list item, or a subsection. Used extensively in grid layouts of cards, capability lists, advisory service listings, etc.

Mobile	(< 768px):	18px
Tablet	(≥ 768px):	20px

.ty-hero — Hero and page descriptions

The paragraph immediately under a page <h1> that explains what the page is about. Also used for CTA descriptions — the sentence under a "Commission Your Research" heading. These are larger than card body text because they're prominent, introductory text.

Mobile	(< 768px):	16px
Tablet	(≥ 768px):	18px

.ty-body — Card body paragraphs

The descriptive text inside a card, below a card heading. The workhorse of content-heavy pages. Also used for list item descriptions, feature explanations, and paragraph text within sections.

Mobile	(< 768px):	14px
Tablet	(≥ 768px):	16px

What @apply does

@apply is a Tailwind directive that expands utility class names into their CSS property values at build time. So:

```
.ty-h1 { @apply text-2xl md:text-3xl lg:text-4xl; }
```

Compiles to:

```
.ty-h1 { font-size: 1.5rem; line-height: 2rem; }
@media (min-width: 768px) {
  .ty-h1 { font-size: 1.875rem; line-height: 2.25rem; }
}
@media (min-width: 1024px) {
  .ty-h1 { font-size: 2.25rem; line-height: 2.5rem; }
}
```

The browser receives standard CSS. It has no knowledge of Tailwind or custom class names.

Why @layer components

Tailwind organizes CSS into three layers with a defined specificity order:

- 1. **@layer base** — Browser resets, global element defaults
- 2. **@layer components** — Named, multi-property classes (like `.ty-h1`)
- 3. **@layer utilities** — Single-purpose utility classes (like `font-black`, `text-slate-900`)

By putting `ty-*` in **@layer components**, Tailwind utility classes that appear alongside them in a `className` string will **always override them**. This is critical:

```
// ty-h1 sets font-size. font-black overrides font-weight. text-rose-600
overrides color.
// All three coexist without conflict.
<h1 className="ty-h1 font-black text-rose-600 mb-6">Custom colored heading</h1>
```

If `ty-h1` were in **@layer utilities**, it would compete with other utility classes and the one that appears last in the CSS would win — unpredictable behavior. **@layer components** prevents this entirely.

6. How to Use the Typography Classes

Basic pattern

```
// ✓ CORRECT – size from ty-*, everything else explicit
<h1 className="ty-h1 font-black tracking-tight text-slate-900 mb-6">
  Vermont Health Platform
</h1>

<p className="ty-hero text-slate-600 leading-relaxed max-w-2xl mb-8">
  Comprehensive healthcare intelligence across six analytical pillars.
</p>

<h3 className="ty-h3 font-bold text-slate-900 mb-3">
  Revenue Cycle Assessment
</h3>

<p className="ty-body text-slate-600 leading-relaxed mb-4">
  End-to-end review from charge capture through final payment.
</p>
```

The mental model

Think of the `ty-*` class as answering **"how big?"** Everything else in the `className` answers **"what color?"**, **"how bold?"**, **"how much space?"**

ty-h1	→ answers: how big is this heading across all screen sizes?
font-black	→ answers: how heavy is the weight?
text-slate-900	→ answers: what color?
tracking-tight	→ answers: what letter spacing?
mb-6	→ answers: how much space below it?

None of these conflict. Each answers a different question.

Using with dynamic colors

The `ty-*` classes only set font size. You can freely combine them with any color:

```
// White heading on dark background
<h1 className="ty-h1 font-black text-white mb-4">Intelligence Platform</h1>

// Pillar-colored heading
<h1 className="ty-h1 font-black text-indigo-700 mb-4">Policy Analysis</h1>

// Rose accent
<h1 className="ty-h1 font-black text-rose-600 mb-4">Advisory Services</h1>
```

Using with template literals (dynamic className)

```
// ✓ Works perfectly – ty-h3 is a plain string, combines freely
<h3 className={`ty-h3 font-bold mb-3 ${isActive ? "text-indigo-700" : "text-slate-900"}`}>
  {card.title}
</h3>

// ✓ Works in conditionals
const headingClass = isPrimary
  ? "ty-h1 font-black text-slate-900"
  : "ty-h2 font-bold text-slate-700";

<h2 className={`${headingClass} mb-6`}>{heading}</h2>
```

Page structure template

Every new page should follow this structure:

```
export default function MyPage() {
  return (
    <div className="bg-white min-h-screen pb-20">

      {/* HERO SECTION */}
      <div className="bg-slate-50 border-b border-slate-200 py-10">
        <div className="container mx-auto px-4 md:px-8 max-w-5xl">
          <h1 className="ty-h1 font-black tracking-tight text-slate-900 mb-4">
            Page Title
          </h1>
          <p className="ty-hero text-slate-600 leading-relaxed max-w-2xl">
            Page description that explains what this page is about.
          </p>
        </div>
      </div>

      {/* CONTENT SECTION */}
      <div className="container mx-auto px-4 md:px-8 max-w-5xl py-16">
        <h2 className="ty-h2 font-bold text-slate-900 mb-2">Section Title</h2>
        <p className="ty-hero text-slate-500 mb-8">Section description.</p>

        <div className="grid md:grid-cols-2 gap-6">
          {items.map((item) => (
            <div key={item.id} className="p-6 border border-slate-200 rounded-
xl">
              <h3 className="ty-h3 font-bold text-slate-900 mb-3">{item.title}
</h3>
              <p className="ty-body text-slate-600 leading-relaxed">
{item.description}</p>
            </div>
          ))}
        </div>
      </div>
```

```
        </div>

    </div>
);
}
```

7. What NOT to Do

Do not write raw responsive text sizes

```
// x WRONG – bypasses the system, breaks consistency
<h1 className="text-2xl md:text-3xl lg:text-4xl font-black text-slate-900">
  Page Title
</h1>

// ✓ RIGHT
<h1 className="ty-h1 font-black text-slate-900">Page Title</h1>
```

If you write the raw classes, a future change to `ty-h1` in `globals.css` will NOT update this heading. You've created a fork that will drift.

Do not use flat (non-responsive) text sizes for headings or body text

```
// x WRONG – no responsive scaling
<h1 className="text-4xl font-black">Title</h1>
<p className="text-lg text-slate-600">Description</p>

// ✓ RIGHT
<h1 className="ty-h1 font-black">Title</h1>
<p className="ty-hero text-slate-600 leading-relaxed">Description</p>
```

The only elements that should use flat text sizes are:

- **Emoji and icon containers** — `text-2xl`, `text-3xl`, `text-5xl` for emoji display
- **Form inputs** — `text-lg` on a search input for legibility
- **Score/metric displays** — `text-5xl font-black` for a large KPI number
- **Labels, badges, metadata** — `text-xs` or `text-sm` (these don't need responsive scaling)

Do not mix `text-gray-*` and `text-slate-*`

This app uses the `slate` color palette exclusively for text. Gray and slate look nearly identical in most cases but produce subtle tone differences at scale, making the app feel inconsistent.

```
// x WRONG
<p className="text-gray-600">Description</p>
<h3 className="text-gray-900 font-bold">Heading</h3>

// ✓ RIGHT
<p className="text-slate-600">Description</p>
<h3 className="text-slate-900 font-bold">Heading</h3>
```

Do not use `font-extrabold` for headings

The app uses `font-black` (weight 900) for primary headings and `font-bold` (weight 700) for secondary headings. `font-extrabold` (weight 800) sits between these and was causing visual inconsistency. It is not used anywhere in the platform.


```
// x WRONG
<h1 className="ty-h1 font-extrabold">Title</h1>

// ✓ RIGHT
<h1 className="ty-h1 font-black">Title</h1>    // primary heading
<h2 className="ty-h2 font-bold">Section</h2>    // secondary heading
```

Do not use **leading-relaxed** on headings

leading-relaxed (line-height: 1.625) is for multi-line paragraph text. Headings should use the default line height that comes with the font size class, or **leading-tight** / **leading-snug** for particularly large headlines.

```
// x WRONG
<h1 className="ty-h1 font-black leading-relaxed">Title</h1>

// ✓ RIGHT
<h1 className="ty-h1 font-black tracking-tight leading-tight">Title</h1>
<p className="ty-body text-slate-600 leading-relaxed">Paragraph.</p>
```

8. How to Change the Scale App-Wide

This is the entire point of the system. To change how all page headings look everywhere:

Open **app/globals.css** and edit the relevant class:

```
/* BEFORE */
.ty-h1 { @apply text-2xl md:text-3xl lg:text-4xl; }

/* AFTER – if you want headings larger on desktop */
.ty-h1 { @apply text-2xl md:text-3xl lg:text-5xl; }

/* AFTER – if you want headings smaller everywhere */
.ty-h1 { @apply text-xl md:text-2xl lg:text-3xl; }
```

Save the file. Every `<h1>` using **ty-h1** across all 200+ pages and components updates instantly on the next build or hot reload. You do not open a single component file.

Example: tightening the body text scale

If user testing shows body text is too large on mobile:

```
/* BEFORE */
.ty-body { @apply text-sm md:text-base; }

/* AFTER – stays at xs on mobile, goes to sm on tablet, base on desktop */
.ty-body { @apply text-xs md:text-sm lg:text-base; }
```

Every **ty-body** usage in every card across every page updates.

9. Adding a New Typography Class

If you have a new, recurring text pattern that doesn't fit the existing six classes, add it to **globals.css** in the same **@layer components** block:

```
@layer components {
  .ty-h1      { @apply text-2xl md:text-3xl lg:text-4xl; }
  .ty-h1-xl   { @apply text-2xl md:text-4xl lg:text-5xl; }
  .ty-h2      { @apply text-xl   md:text-2xl lg:text-3xl; }
  .ty-h3      { @apply text-lg   md:text-xl;           }
  .ty-hero    { @apply text-base md:text-lg;           }
  .ty-body    { @apply text-sm   md:text-base;         }

  /* NEW — for sidebar navigation items */
  .ty-nav     { @apply text-xs   md:text-sm; }
}
```

Rules for adding a new class:

1. It must represent a **recurring pattern** — not a one-off. If it's used in fewer than 5 places, just write the raw classes for now.
2. It must be **strictly about font size** — no colors, no weights, no spacing. Those stay at the usage site.
3. Name it with the **ty-** prefix so it's instantly recognizable as part of this system.
4. Document what it's for in the comment block above the class list.

10. Color, Weight, and Spacing — The Rules

The **ty-*** classes handle font size only. Everything else follows these conventions:

Font weight

Usage	Class	Weight
Primary page <h1> headings	font-black	900
Section <h2> headings	font-bold	700
Card <h3> headings	font-bold	700
Body paragraphs	(default)	400
Labels, overlines, badges	font-bold	700
Navigation, UI labels	font-semibold	600

Text color (light mode)

Usage	Class
Primary headings	text-slate-900
Secondary headings	text-slate-800
Body text	text-slate-600
Secondary / muted text	text-slate-500
Disabled / hint text	text-slate-400
White text on dark backgrounds	text-white
Inverted text on colored bg	text-slate-100

Line height

Usage	Class	When to use
Multi-line body text	leading-relaxed	Paragraphs, descriptions, card body
Large display headings	leading-tight	H1s with tracking-tight

Usage	Class	When to use
Compact headings	<code>leading-snug</code>	H2/H3 in tight layouts
Single-line UI labels	(default)	Buttons, badges, nav items

Letter spacing

Usage	Class
Large <code>ty-h1</code> headings	<code>tracking-tight</code>
Section headings	(default)
Labels, overlines, badges	<code>tracking-widest</code>
Body text	(default)

11. Dark Mode

The `ty-*` classes set font size only — they are color-agnostic. Dark mode text colors are handled by adding `dark:` variants alongside the light mode color:

```
// Light: slate-900 | Dark: slate-100
<h1 className="ty-h1 font-black text-slate-900 dark:text-slate-100 mb-6">
  Platform Title
</h1>

// Light: slate-600 | Dark: slate-400
<p className="ty-body text-slate-600 dark:text-slate-400 leading-relaxed">
  Description text.
</p>
```

This is explicit and intentional — the dark mode color for each element is visible at the usage site, not hidden in a global rule. This makes it easy to read, debug, and override.

12. Emoji and Icon Sizing — The Exception

Emoji containers, icon wrappers, and score display numbers are the **only** legitimate use of flat (non-responsive) large text sizes. These are presentational elements, not text content, and their size is a deliberate design choice rather than a content scale decision.

```
// ✓ CORRECT — emoji display, not a heading
<div className="text-5xl mb-4">🚀</div>

// ✓ CORRECT — large KPI metric number
<div className="text-5xl font-black text-emerald-600">{score}</div>

// ✓ CORRECT — icon container with background
<div className="w-12 h-12 rounded-lg flex items-center justify-center text-2xl
bg-indigo-100">
  💡
</div>

// ✗ WRONG — this is an actual heading, must use ty-h1
<h1 className="text-5xl font-black">Page Title</h1>
```

The distinction: if a human would read the content as a heading or body text, use `ty-*`. If it is a visual/decorative display element (emoji, icon, number badge), a flat size is fine.

13. Quick Reference Card

Copy this and keep it visible while building pages.

HEADINGS

ty-h1	Page <h1>	2xl → 3xl → 4xl
ty-h1-xl	Large marketing hero	2xl → 4xl → 5xl
ty-h2	Section <h2>	xl → 2xl → 3xl
ty-h3	Card/sub <h3>	lg → xl

BODY TEXT

ty-hero	Hero/page descriptions	base → lg
ty-body	Card body paragraphs	sm → base

FONT WEIGHT

font-black	→ primary h1 headings (900)
font-bold	→ section h2, card h3, labels (700)
font-semibold	→ nav items, UI labels (600)
(default)	→ body paragraphs (400)

TEXT COLOR

text-slate-900	primary headings
text-slate-600	body text
text-slate-500	secondary / muted
text-slate-400	disabled / hint
text-white	headings on dark backgrounds

LINE HEIGHT

leading-relaxed	→ paragraphs and descriptions
leading-tight	→ large display headings
(default)	→ UI labels, buttons

EXAMPLE: CARD

```
<div className="p-6 border border-slate-200 rounded-xl">
  <h3 className="ty-h3 font-bold text-slate-900 mb-3">Card Title</h3>
  <p className="ty-body text-slate-600 leading-relaxed">{description}</p>
</div>
```

EXAMPLE: PAGE HERO

```
<h1 className="ty-h1 font-black tracking-tight text-slate-900 mb-4">
  Page Title
</h1>
<p className="ty-hero text-slate-600 leading-relaxed max-w-2xl mb-8">
  Page description.
</p>
```

NEVER DO THIS

x text-2xl md:text-3xl lg:text-4xl	← write ty-h1 instead
x text-gray-*	← write text-slate-* instead
x font-extrabold	← write font-black or font-bold
x text-4xl (flat, no md:/lg:)	← unless it's an emoji/icon/number

